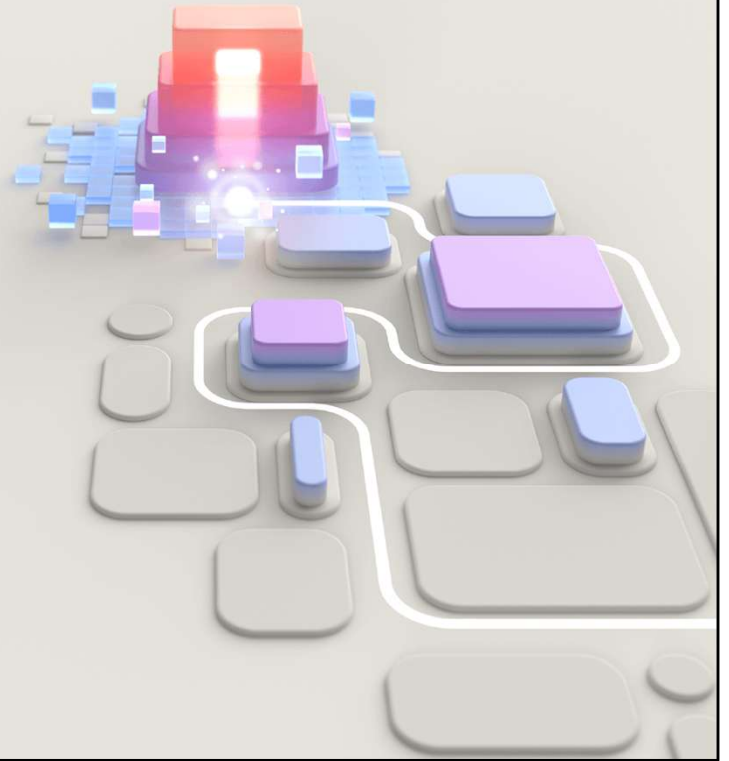




# DP-300

## Implement scalable database solutions using Azure SQL



© Copyright Microsoft Corporation. All rights reserved.

# Monitor and optimize operational resources in Azure SQL

<https://learn.microsoft.com/training/paths/monitor-optimize-operational-resources-sql-server/>

© Copyright Microsoft Corporation. All rights reserved.

We'll teach you about resource optimization for your databases created using either IaaS or PaaS services. We'll also cover monitoring server and hardware resources. It will familiarize you with the various tools available for monitoring performance and establishing a baseline. You will learn how to interpret performance metrics for the most critical resources. You will also learn how to troubleshoot database performance using Azure Intelligent Insights.

<https://learn.microsoft.com/training/paths/monitor-optimize-operational-resources-sql-server/>

## Objective



Monitor activity and compare to a baseline



Explore maintenance tasks related to performance



Identify major causes of performance problems



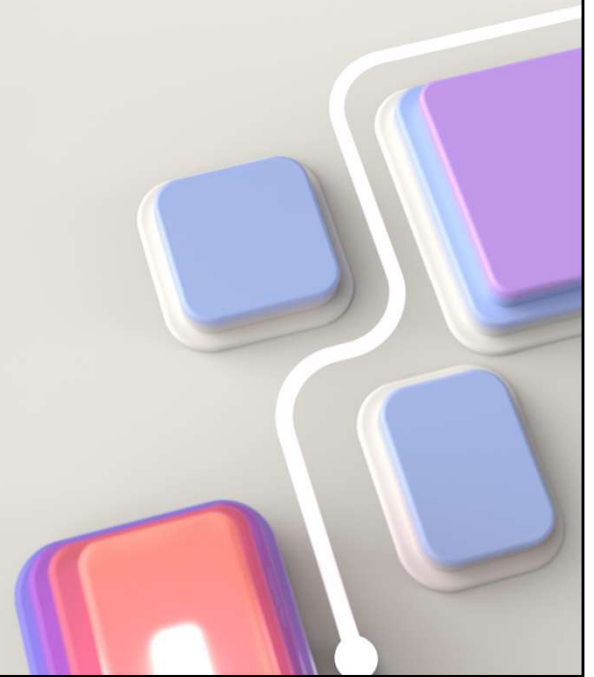
Configure databases and resources for optimal performance

© Copyright Microsoft Corporation. All rights reserved.

# Describe performance monitoring

<https://learn.microsoft.com/training/modules/describe-performance-monitoring/>

© Copyright Microsoft Corporation. All rights reserved.



# Objectives



Understand methods to review potential performance issues



Learn how to collect metrics for an established baseline



Understand Extended Events and Azure Intelligent Insights

© Copyright Microsoft Corporation. All rights reserved.

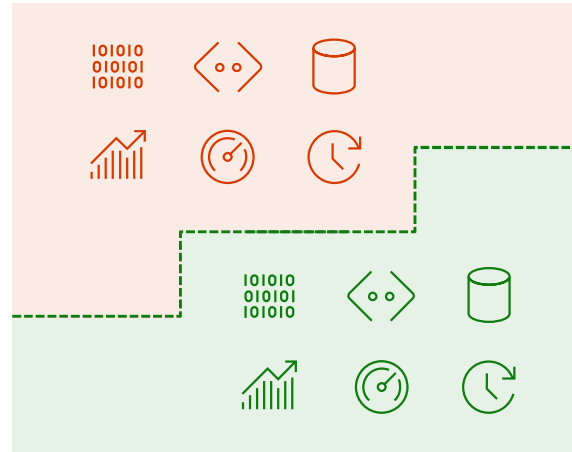
We'll compare Azure, and on-premises tools for monitoring and measuring performance. Determine critical metrics. Understand the purpose of a baseline for comparative analysis. Configure extended event sessions for tracing activities.

# What is a baseline?

## A baseline

A general set of performance and utilization metrics that helps distinguish anomalies from typical usage patterns.

Baseline



© Copyright Microsoft Corporation. All rights reserved.

A baseline is a collection of metric data that helps you understand the normal “steady state” of your application or server’s performance. Having the data collected over time allows you to identify changes from normal state. Baselines can be as simple as a chart of CPU utilization over time, or complex aggregations of metrics to offer granular level performance data from specific application calls. The granularity of your baseline will depend on the criticality of performance of your database and application.

The metrics can be as simple as CPU and Disk utilization or collect a detailed set of metrics on OS and database utilization

## Establishing a baseline

- Having a baseline of performance enables better decisions for performance tuning
- It helps distinguish code issues from increased utilization
- Allows proactive allocation of additional compute resources as workloads grow.

© Copyright Microsoft Corporation. All rights reserved.

With any type of application workload, it is imperative to establish a working baseline. A baseline will help you identify if an ongoing issue should be considered within normal parameters or has exceeded given thresholds. Without a baseline, every issue encountered could be considered normal and therefore not require any additional intervention.

Azure Monitor is just one example of a tool that allows you to build a baseline for system performance for a given system.

# Performance on an Azure Virtual Machine

## Built-in tool in the Windows operating system

Collect detailed, granular performance data about SQL Server and Windows operations

Group Performance Monitor counters together to easily gather and analyze performance data

Data can be easily exported to archive

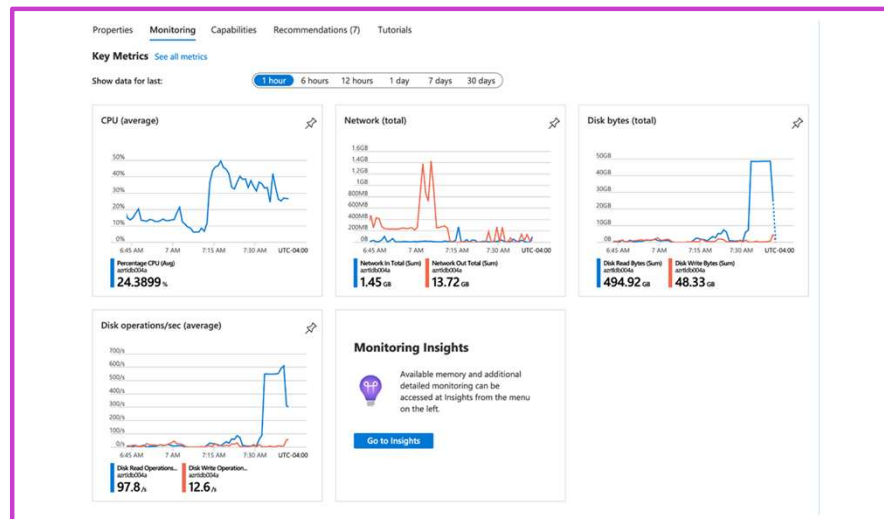
© Copyright Microsoft Corporation. All rights reserved.

Whether you using on-premises server or on an Azure virtual machine, the Windows Server platform has a native tool called Performance Monitor (commonly shortened to perfmon after the name of its executable file) that allows you to easily and routinely monitor performance metrics. Perfmon operates with counters for both the operating systems and installed program. When SQL Server is installed on the operating system the database engine creates its own group of specific counters.

- Performance Monitor allows you to collect detailed, granular performance data about SQL Server and Windows operations.
- You can group Performance Monitor counters together to easily gather and analyze performance data
- Data can be easily exported to archive (e.g. a SQL Server database).

# Performance monitoring tools – Azure Monitor

- All Azure resources collect a set of metrics through the Azure Monitor service
- Enhanced data may be collected through the Azure Monitoring Insights for virtual machine resources
- Data is stored in Azure Log Analytics
- Metrics available to monitor will vary depending on the type of resource



© Copyright Microsoft Corporation. All rights reserved.

- Azure Monitor är en central tjänst för att övervaka prestanda och hälsa hos alla dina Azure-resurser. Varje resurs i Azure – oavsett om det är virtuella maskiner, databaser eller webbtjänster – samlar in olika mätvärden som CPU-användning, nätverkstrafik, diskoperationer med mera. Dessa mätvärden samlas och visas genom Azure Monitor. För virtuella maskiner kan man få fördjupad data via Azure Monitoring Insights, som ger en ännu mer detaljerad bild av resursanvändningen. All denna data kan lagras i Azure Log Analytics, vilket ger möjlighet att utföra avancerade analyser och skapa rapporter. Mätvärdena varierar beroende på vilken typ av resurs du övervakar; en databas har andra relevanta mätvärden än en VM. Med Azure Monitor kan du även sätta upp varningar baserade på tröskelvärden, vilket gör det enkelt att agera i tid om något avviker från det normala. Tjänsten ger en visuell översikt i form av grafer och dashboards, där du kan se trender och historik. Sammanfattningsvis är Azure Monitor ett kraftfullt verktyg som hjälper dig att proaktivt följa och optimera dina Azure-resurser, så att du kan säkerställa hög tillgänglighet och prestanda i dina applikationer och tjänster.

- The data displayed here is the default metrics captured for an Azure VM operating system.
- Azure Virtual Machines from the Azure marketplace, have an agent installed automatically in the VM that has a basic set of operating system metrics.
- Data is stored in Azure Log Analytics and is queried by a SQL-like language called Kusto Query Language (KQL).

The following set of examples is focused on an Azure Virtual Machine. When you deploy an Azure Virtual Machine from the Azure marketplace, an agent is installed in the VM which provides a baseline set of operating system metrics that are presented to you in the Azure portal. This agent provides metrics into a service called Azure Monitor which is a comprehensive platform monitoring solution that collects and displays a standard set of metrics from Azure resources. In the case of a VM, by default CPU, network utilization, and disk read and write operations.

These Azure Monitor Insights can be one way of viewing a baseline of performance for your Azure VM including I/O performance, memory and CPU utilization. This data is stored in an Azure Log Analytics workspace. Azure Log Analytics is the primary tool in Azure for storing and querying log files of all kinds. Log Analytics is queried by a SQL-like language called Kusto Query Language (KQL).

## Review of Azure metric alerts

| Overview                                                                                                                                                      | Creating metric alerts                                                                                                            |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Azure has a built-in alerting system that allows you to create alerts                                                                                         | Metric alerts can be defined on IaaS or PaaS resources                                                                            |
| The alerts can be configured: <ul style="list-style-type: none"> <li>• In a static manner</li> <li>• In a dynamic fashion using Dynamic Thresholds</li> </ul> | Alerts can be scoped by: <ul style="list-style-type: none"> <li>• VM</li> <li>• Subscription</li> <li>• Resource Group</li> </ul> |

© Copyright Microsoft Corporation. All rights reserved.

The Azure Monitor service includes the ability to track various metrics about the overall health of a given resource. Metrics are gathered at regular intervals and are the gateway for alerting processes that will help to resolve issues quickly and efficiently. Azure Monitor Metrics is a powerful subsystem that allows you to not only analyze and visualize your performance data, but to also trigger alerts that notify administrators or automated actions that can trigger an Azure Automation runbook or a webhook. You also have the option to archive your Azure Metrics data to Azure Storage for archiving, since active data is kept online stored for 93 days.

- These alerts can simply be notifications like email or text message, or can be automated actions like executing an **Azure Function** or an **Azure Automation runbook**.
- The alerts can be configured based on the metrics in your VM or other Azure resource
- You can also configure dynamic alerts which learn the behavior of your resource and only alert on abnormal usage

The alerts can be configured in a static manner (e.g. alert when CPU goes over 95%) or in a dynamic fashion using Dynamic Thresholds. Dynamic Thresholds learn the historical behavior of the metric and alert when the resources are operating in an abnormal basis. This can detect seasonality in your workloads and adjust the alerting accordingly

### Creating metric alerts

Alerts are grouped by region but can be scoped by:

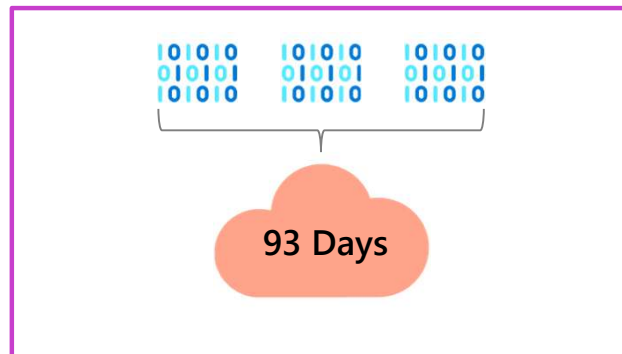
- VM
- Subscription
- Resource Group

Alerts may be created on a number of metrics that are defined within Azure Monitor

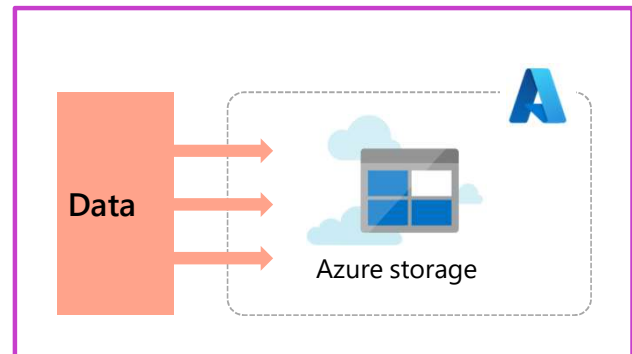
Alert targets include email, SMS, Azure Automation, Functions, and Logic Apps

Utilizing the Azure portal, you can create alert rules, based on defined metrics, in the overview section of the Azure Monitor blade. Azure Monitor Alerts can be scoped in three ways. For example, using Azure Virtual Machines as an example you can specify the different scopes (all resources need to be in the same Azure region), but you can include all of the machines in a subscription, VMs in one or more resource groups, or all of the VMs in the entire sub.

## Reviewing performance data history



Azure Monitor stores data from **93 days**, which covers three months of data



You may archive data to Azure storage for historical analysis

© Copyright Microsoft Corporation. All rights reserved.

One of the benefits of utilizing the Azure Monitor is the ability to easily and quickly review past metrics that were gathered. If you examine a resource, you'll note a datetime picker in the upper right-hand corner, as seen in Figure 12. Azure Monitor Metrics will be retained for 93 days, after which they are purged, however you do have the option to archive them to Azure Storage.

## SQL Server metrics that matter

|                                                                           |                               |
|---------------------------------------------------------------------------|-------------------------------|
| Processor(_Total)% Processor Time                                         | Paging File(_Total)% Usage    |
| PhysicalDisk(_Total)\Avg. Disk sec/Read and Avg. Disk sec/Write           | System\Processor Queue Length |
| SQLServer:Buffer Manager\Page life expectancy                             |                               |
| SQLServer:SQL Statistics\Batch Requests/sec                               |                               |
| SQLServer:SQL Statistics\SQL Compilations/sec and SQL Re-Compilations/sec |                               |

© Copyright Microsoft Corporation. All rights reserved.

### **Processor(\_Total)% Processor Time**

Mäter total CPU-användning på servern. Bra indikator på arbetsbelastning och kan, med andra mått, avslöja query-problem.

### **Paging File(\_Total)% Usage**

SQL Server ska normalt inte behöva page:a. Hög paging beror ofta på andra tjänster och ger tydlig prestandaförsämring.

### **PhysicalDisk(\_Total)\Avg. Disk sec/Read & Avg. Disk sec/Write**

Visar disk-latens och lagringsprestanda. Bör oftast vara under 20 ms, Premium Storage helst under 10 ms.

### **System\Processor Queue Length**

Antal trådar som väntar på CPU-tid. Värdet över noll tyder på CPU-press och behov av fler CPU:er.

### **SQLServer:Buffer Manager\Page life expectancy**

Hur länge sidor förväntas ligga i minnet. Beakta trender och plötsliga dropp; kan tyda på minnespress eller dåliga queries.

### **SQLServer:SQL Statistics\Batch Requests/sec**

Visar hur konstant upptagen SQL Server är. Inga fasta riktvärden, men jämför över tid och mot CPU för baseline.

### **SQLServer:SQL Statistics\SQL Compilations/sec & SQL Re-Compilations/sec**

Många kompileringar betyder att planer saknas eller invalideras. Kan bero på ad-hoc queries, RECOMPILE-hints eller plan-cache minnespress.

When deploying SQL Server on an Azure virtual machine, it's critical to correlate the performance of SQL Server with that of the underlying operating system. If you are using Linux as the operating system, you will need to install InfluxDB, collectd, and Grafana to capture similar data to Windows Performance Monitor. These services collect data from SQL Server and provide a graphical interface to review the data. Utilizing these tools on Linux or Performance Monitor on Windows in conjunction with understanding how to interrupt SQL Server wait statistics will allow you to identify bottlenecks in hardware or code

**Processor(\_Total)\% Processor Time** - This counter simply measures the CPU utilization of all of the

processors on the server. It is a good indication of the overall workload, and when used in conjunction with other counters, can identify problems with query performance

**Paging File( Total)\% Usage** - In a properly configured SQL Server memory should not page to the paging file on disk. However, in some configurations you may have other services running that consume system memory, and lead to the operating system paging memory to disk resulting in performance degradation.

**PhysicalDisk( Total)\Avg. Disk sec/Read and Avg. Disk sec/Write** - This is a good metric for how the storage subsystem is working. Your latency values in most cases should not be above 20ms, and with Premium Storage you should see values less than 10ms.

**System\Processor Queue Length** - This number indicates the number of threads that are waiting for the time on the processor. If it is greater than zero, it indicates CPU pressure, indicating your workload would benefit from more CPUs.

**SQLServer:Buffer Manager\Page life expectancy** - Page life expectancy indicates how long SQL Server expects a page to live in memory. There is no proper value for this setting (older documentation refers to 300 seconds as proper, but that was written in a 32-bit era when servers had far less RAM). You should monitor this over time, and evaluate sudden drops. This could indicate poor query patterns, external memory pressure (the server running a large SSIS package) or could just be normal system processing like running a consistency check on a large database.

**SQLServer:SQL Statistics\Batch Requests/sec** - This counter is good for evaluating how consistently busy a SQL Server is over time. Once again there is no good or bad value, but you can use this in conjunction with % Processor time to better understand your workload and baselines.

**SQLServer:SQL Statistics\SQL Compilations/sec and SQL Re-Compilations/sec** - These counters will be triggered when SQL Server has to compile or recompile an execution plan for a query because there is no existing plan in the plan cache, or because a plan was invalidated because of a change. This can indicate T-SQL with recompile query hints, or indicative of memory pressure on the plan cache caused by either a lot of ad-hoc queries or simple memory pressure.

# What are Extended Events?

**A lightweight and extensive diagnostic system that is built into the SQL Server engine**

Extended Events are supported on all the Azure SQL platforms

Event sessions can be used to trace activity within the database which can help you troubleshoot issues like:

- Blocking and deadlocking performance issues
- Identify long-running queries
- Observing Memory Pressure in your database

The Extended Event framework allows you to filter your data collection

© Copyright Microsoft Corporation. All rights reserved.

Extended events are a lightweight and very extensive diagnostic system that is built into SQL Server and Azure SQL Database and Managed Instance. Extended Events allow you collect additional information about the internal operations of your databases. Historically, DBAs used a tool call Profiler to trace inbound queries and gather execution plans to identify to problematic queries and their execution plans. Extended Events builds on the functionality of Profiler by allowing you to trace queries and by exposing additional data (events) that you can monitor. Some examples of issues you might troubleshoot with Extended Events include:

- Troubleshooting blocking and deadlocking performance issues
- Identifying long-running queries
- Monitoring Data Definition Language (DDL) operations
- Logging missing column statistics
- Observing Memory Pressure in your database
- Long-running physical I/O operations

The extended event framework also allows you to use filters to limit the amount of data you collect in order to reduce the overhead of data collection, and allows you to more easily identify your performance problem by targeting your focus onto specific areas

## What can I monitor with Extended Events?

- Extended events cover all aspects of the database engine. Specifically they are divided into four channels:

### Admin

Events targeted for end users and administrators, like a deadlock report

### Operational

Used for analysis and diagnosis of common problems, like an Availability Group failover

### Analytical

Commonly used for tracing query execution

### Debug

Used in conjunction with Microsoft support. May not be fully documented

- Events can be groups together into sessions for easier monitoring

© Copyright Microsoft Corporation. All rights reserved.

Extended Events cover the full surface area of SQL Server, and are divided into four channels, which define the audience of an event.

**Admin** - Admin events are targeted for end users and administrators. The events included indicate a problem within a well-defined set of actions an administrator can take. An example of this is the generation of an XML deadlock report to help identify the root cause of the deadlock.

**Operational** - Operational events are used for analysis and diagnostics or common problems. These events can be used to trigger an action or task based on an occurrence of the event. An example of an operational event would be a database in an availability group changing state, which would indicate a failover.

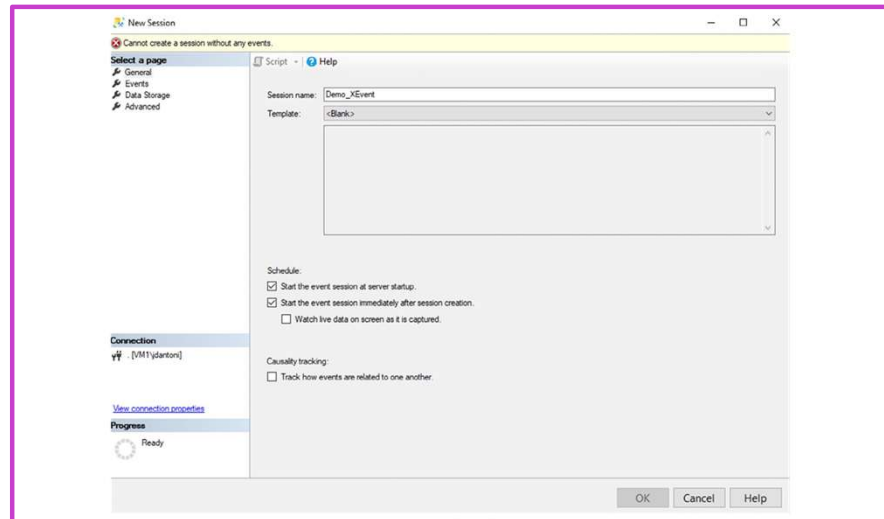
**Analytic** - Analytic events are typically related to performance events and are published in high volume. Tracing stored procedure or query execution would be an example of an analytic event.

**Debug** - Debug events are not necessarily fully documented and should only be used when troubleshooting in conjunction with Microsoft support.

Events are added to sessions which can host multiple events. Typically, multiple events are grouped together in a session to capture a related set of information.

## Create an Extended Events session

- SSMS has a wizard to create an Extended Events session
- SSMS also includes predefined templates for monitoring specific resources
- You can also use T-SQL which is more common for advanced administrators



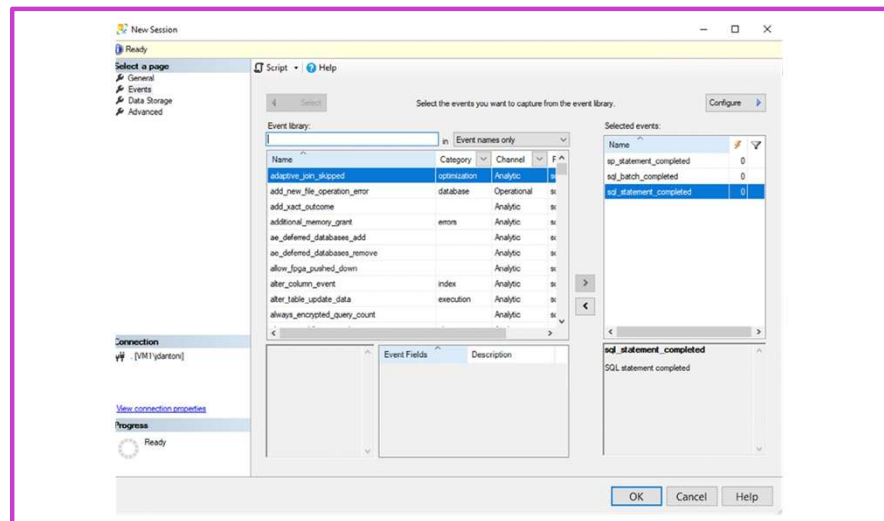
© Copyright Microsoft Corporation. All rights reserved.

Here you will see the basic process of creating an Extended Events session using the New Session dialog from SQL Server Management Studio. You can get to this screen by expanding the Management node in SSMS, expanding the Extended Events node, right-clicking on Sessions and selecting New Session.

The above statement will generate a list of T-SQL statements to backup all of the database on the server. Typically, this generated T-SQL will be executed using `sp_executesql` or passed to another program to execute.

## Extended Events components

- Event Sessions consists of events and targets
- Events represent the engine action you are capturing and include fields that can both be filtered and select individually
- There are multiple targets depending on the nature of your data collection
- The most used storage target is the event file, which writes the file to a local disk



© Copyright Microsoft Corporation. All rights reserved.

An event represents a point of interest within the code the of database engine—these can represent purely internal system operations, or they can be associated with user actions like query execution. In the above example, you can see that three events (`sp_statement_completed`, `sql_batch_completed`, `sql_statement_completed`) have been added to this event session. By default, this session would capture all instances of these events taking place on your instance. You can limit collection by clicking the configure button.

Filters are a powerful feature of Extended Events that allow you to use granular control to capture only the specific occurrences of the event you want to capture. In this example, you can see that filter is being applied on the field `sqlserver.is_system` where it is equal to zero, which indicates that the query is not an internal operation. In other words, the session will not capture completion of statements that are submitted by system connections. We only want to capture statements submitted by users or user applications.

## Extended Events with T-SQL

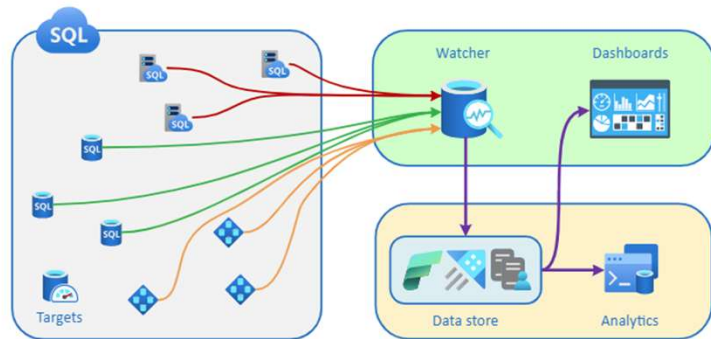
```
CREATE EVENT SESSION test_session
ON SERVER
    ADD EVENT sqllos.async_io_requested,
    ADD EVENT sqlserver.lock_acquired
    ADD TARGET package0.etw_classic_sync_target
    (SET default_etw_session_logfile_path =
     N'C:\demo\traces\sqletw.etl' )
WITH (MAX_MEMORY=4MB, MAX_EVENT_SIZE=4MB);
GO
```

© Copyright Microsoft Corporation. All rights reserved.

T-SQL is the preferred way to create an Extended Events session, due to its simplicity and repeatability. Event sessions can be scoped to a server or a database. In the example shown above, you are adding two events and using the Event Tracing for Windows (ETW) path, with a file location. After you create the session, you'll have to start it. You can do this using T-SQL and ALTER the session using the STATE option, or you can use the Extended Events | Session node in SQL Server Management Studio. You can also have the session start at server startup, which is common for lightweight event sessions that monitor over time.

## Database watchers

- 1 In the Azure portal, search for "Database Watchers" and select it. Select **Create**.
- 2 Select your subscription and resource group. Provide a name, and select the region where you want to deploy the database watcher.
- 3 On the **Identity** tab, choose an identity to authenticate to SQL targets, key vault, and data store.
- 4 On the **Data store** tab, configure the data store for the monitoring data.
- 4 On the **SQL targets** tab, choose the Azure SQL Database or Azure SQL Managed Instance you want to monitor.



© Copyright Microsoft Corporation. All rights reserved.

Bilden beskriver hur man använder **Database Watchers** i Azure för att övervaka flera SQL-mål på ett samlat sätt. En Database Watcher är en Azure-resurs som kan kopplas till en eller flera databaser, exempelvis **Azure SQL Database** eller **Azure SQL Managed Instance**, och samla in övervakningsdata för analys och dashboards.

Processen börjar i Azure-portalen där man söker efter **Database Watchers** och väljer **Create**. Därefter väljer man subscription och resource group, anger ett namn och väljer den region där watcher-resursen ska skapas. I nästa steg, under fliken **Identity**, väljer man vilken identitet som ska användas för autentisering mot SQL-målen, Key Vault och datalagringen. Detta gör att man kan hantera åtkomst på ett säkert sätt utan att lagra lösenord.

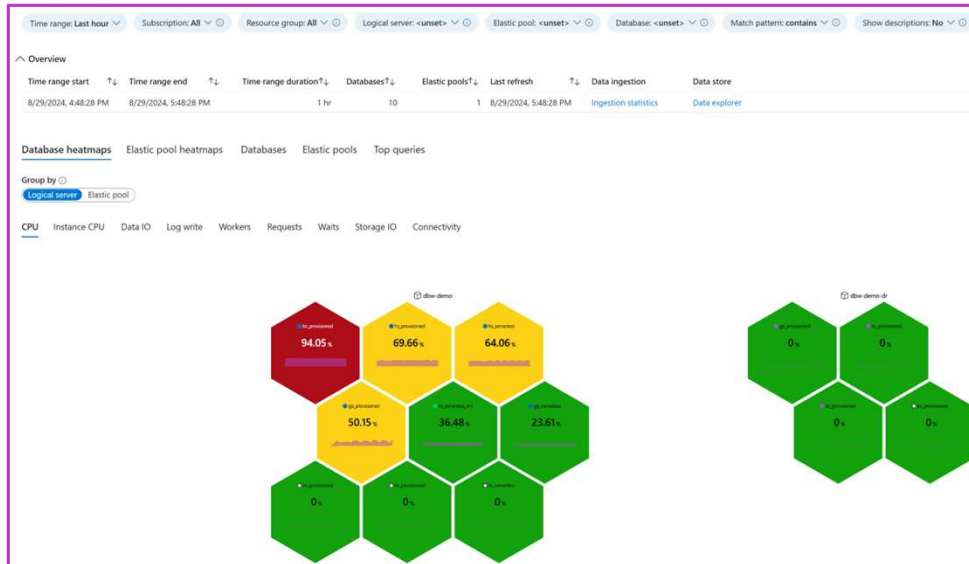
På fliken **Data store** konfigurerar man var insamlad övervakningsdata ska lagras. Watchern skickar sedan data vidare till analys- och rapporteringslösningar, exempelvis logganalys eller andra analytics-tjänster. Slutligen väljer man under fliken **SQL targets** vilka databaser eller instanser som ska övervakas.

Sammanfattningsvis fungerar Database Watchers som en central "hub" som samlar in data från flera SQL-resurser och gör det lättare att visualisera prestanda och avvikelser i dashboards.

One of the benefits of using any of the products part of the Azure SQL family is the monitoring capability that is built into Azure platform.

[Database watcher](#) is a robust monitoring solution designed specifically for [Azure SQL Database](#) and [Azure SQL Managed Instance](#). This tool provides comprehensive insights into the performance, configuration, and overall health of your databases. It collects detailed monitoring data from various sources, including databases, elastic pools, and SQL managed instances, and ensures that you have a clear and detailed view of your SQL estate.

# Database watchers heatmap



© Copyright Microsoft Corporation. All rights reserved.

Absolut – den här bilden visar **Database Watchers heatmap** i Azure-portalen. Så här får du fram den steg för steg:

## 1) Skapa/ha en Database Watcher

Först måste du ha en **Database Watcher** som övervakar minst en databas.

I Azure-portalen: sök efter **Database Watchers**

Öppna din watcher

Kontrollera att du har:

- valt **SQL targets** (databaser/MI)
- konfigurerat **Data store**
- valt **Identity** med rätt behörigheter

Utan detta finns ingen data → ingen heatmap.

## 2) Öppna Watcher-resursen

Gå till Azure-portalen

Sök: **Database Watchers**

Klicka på din watcher-resurs

## 3) Gå till Heatmap-vyn

I watcher-menyn (eller i flikarna i själva vyn) välj:

**Database heatmaps**

(ibland står det "Heatmaps" eller "Database heatmaps" beroende på portal-version)

Det är exakt den vy som syns i bilden.

**4) Välj tidsintervall och filtrering**

Överst i heatmap-vyn finns filter som i bilden:

**Time range** (t.ex. Last hour / 24 hours / 7 days)

**Subscription**

**Resource group**

**Logical server**

**Elastic pool**

**Database**

ev. match pattern / contains

För att få fram hexagonerna måste du välja ett tidsintervall där det finns insamlad data.

**5) Välj vad heatmapen ska visa**

Under "Database heatmaps" kan du välja typ av mätvärde, t.ex.:

**CPU**

Instance CPU

Data IO

Log write

Workers

Requests

Waits

Storage IO

Connectivity

I bilden är **CPU** vald.

**6) Tolka färgerna**

**Grön** = låg belastning / inga problem

**Gul** = medel / varning

**Röd** = hög belastning / risk för throttling eller prestandaproblem

Varje hexagon representerar en databas (eller pool beroende på group by).

Vill du att jag skriver en kort "slide-text" (2–3 bullets) som beskriver detta för kursmaterial?

Database watcher uses [Azure Workbooks](#) to provide monitoring dashboards at the estate level and at the resource level.

Here is an example of a database CPU utilization heatmap on the estate dashboard. Each hexagon represents a SQL target. There are two logical servers, one with six databases and one with three databases. The high availability secondary replicas are shown on the heatmap as separate targets. Select the image to see additional details, including data ingestion statistics.

# Query Performance Insights

## Query Performance Insight Allows you...

- Allows DBAs to quickly identify expensive queries

- You can drill into individual queries

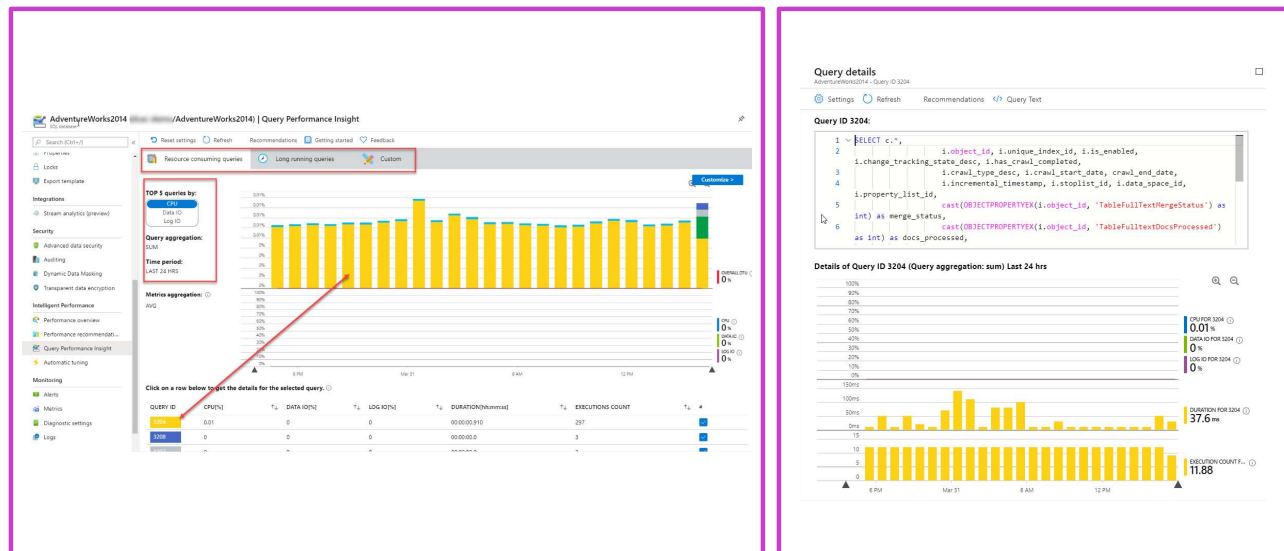
- You can combine the Query Performance Insight dashboard with Query Store data to quickly identify the most expensive execution plan

© Copyright Microsoft Corporation. All rights reserved.

Identifying which queries are consuming the most resources is the first step in any database performance tuning endeavor. In older versions of SQL Server, this required extensive tracing and a series of complex SQL scripts, which could make the process of data gathering cumbersome.

Azure SQL Database offers a tool called Query Performance Insight, that allows the administrator to quickly identify expensive queries. You can navigate to Query Performance Insight in the main blade for your Azure SQL Database in the Intelligent Performance section.

# Query Performance Insights



© Copyright Microsoft Corporation. All rights reserved.

Azure SQL Database offers a tool called Query Performance Insight, that allows the administrator to quickly identify expensive queries. You can navigate to Query Performance Insight in the main blade for your Azure SQL Database in the Intelligent Performance section.

When you launch Query Performance Insight, you'll discover three buttons to allow you to filter for long running queries, top resource consuming queries, or a custom filter. The default value is Resource Consuming Queries. This tab will show you the top five queries sorted by the particular resource that you select on the left. In this case, it was sorted by CPU. You also have other options of sorting by Data IO and Log IO metrics.

You can drill into individual queries by clicking on the row within the lower grid. Each row will be identified with a unique color that correlates to the color within the bar graph above it.

## Demo: Query Performance Insight

**Showcase Query Performance  
Insight in the Azure portal**

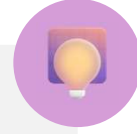
© Copyright Microsoft Corporation. All rights reserved.

Demo Setup: Run a number of queries against a database in Azure SQL Database

Demo: Navigate to Query Performance Insight for your SQL database and walk the students through identifying expensive queries using the tool

<https://github.com/roblevay/dp300/QueryPerformanceInsight.sql>

## Knowledge check



Which data store options can be used to analyze SQL monitoring data collected by Database Watcher?

- ☒ Azure Data Explorer
- ☐ Azure Blob Storage
- ☐ Azure Event Grid



Which Extended Events target only writes to memory and is not persisted?

- ☒ Ring Buffer
- ☐ Target File
- ☐ Event Tracking for Windows



Which Performance Monitor counter reflects how long SQL Server expects to retain data in memory?

- ☐ Processor Queue Length
- ☒ Page Life Expectancy
- ☐ Paging File Usage

© Copyright Microsoft Corporation. All rights reserved.

**1) Which data store options can be used to analyze SQL monitoring data collected by Database Watcher?**

☒ **Rätt: Azure Data Explorer**

*Database Watcher lagrar och analyserar data effektivt i ADX.*

☒ **Fel: Azure Blob Storage**

*Blob kan lagra filer, men är inte analysmotorn här.*

☒ **Fel: Azure Event Grid**

*Event Grid är för händelserouting, inte datalagring/analys.*

**2) Which Extended Events target only writes to memory and is not persisted?**

☒ **Rätt: Ring Buffer**

*Ring buffer lagrar events i minne och skrivs inte till disk.*

☒ **Fel: Target File**

*Event file sparar data på disk, alltså persistent lagring.*

☒ **Fel: Event Tracking for Windows**

*ETW är Windows-spårning, inte XEvent-target i SQL.*

**3) Which Performance Monitor counter reflects how long SQL retains data in memory?**

☒ **Fel: Processor Queue Length**

*Mäter CPU-köer, inte hur länge data stannar i minnet.*

☒ **Rätt: Page Life Expectancy**

*Visar hur länge sidor förväntas ligga kvar i buffer cache.*

☒ **Fel: Paging File Usage**

*Mäter OS paging till disk, inte SQL Servers cache-retention.*



## Instructor led labs: Isolate performance problems through monitoring

**Review CPU utilization in Azure portal**  
**Identify high CPU queries**

© Copyright Microsoft Corporation. All rights reserved.

<https://microsoftlearning.github.io/dp-300-database-administrator/Instructions/Labs/06-isolate-performance-problems.html>

# Configure SQL Server resources for optimal performance

<https://learn.microsoft.com/training/modules/configure-sql-server-resources-optimal-performance/>

© Copyright Microsoft Corporation. All rights reserved.

One of the challenges administrators face in the cloud is balancing costs and performance. Both Azure and SQL Server provide many options for configuration to meet the needs of small and large workloads. Choosing the right storage and sizing your virtual machine are critical steps in meeting the performance needs of your applications and balancing cloud costs. Proper configuration of SQL Server resources like TempDB which can easily become a performance bottleneck, and Resource Governor, which can be used to manage multi-tenant workloads, are also important for properly maintaining your server performance.

## Objectives



Understand your options for configuration of Azure Storage



Learn how to configure TempDB data files in SQL Server



Learn how to choose the right type of VM for SQL Server workloads



Understand the use cases and configuration of Resource Governor in SQL Server

© Copyright Microsoft Corporation. All rights reserved.

One of the challenges administrators face in the cloud is balancing costs and performance. Both Azure and SQL Server provide many options for configuration to meet the needs of small and large workloads. Choosing the right storage and sizing your virtual machine are critical steps in meeting the performance needs of your applications and balancing cloud costs. Proper configuration of SQL Server resources like TempDB which can easily become a performance bottleneck, and Resource Governor, which can be used to manage multi-tenant workloads, are also important for properly maintaining your server performance.

# Azure Storage

Azure Storage is a scalable, secure storage platform that meets a variety of application data storage needs beyond just VM storage.

SQL Server can use three types of Azure Storage:

## Blob Storage

Can be used for database backups

## File Storage

File shares that can be used for Failover Cluster Instances

## Disk Storage

Managed block storage that is used to provide storage to VMs

© Copyright Microsoft Corporation. All rights reserved.

### Blob Storage → Databasbackuper

Blob Storage passar för backuper eftersom det är **billig och skalbar objektlagring** för stora filer. Det är byggt för att lagra backup-filer säkert, med hög hållbarhet och enkel åtkomst via URL/SAS.

### File Storage → Failover Cluster Instances (FCI)

File Storage (Azure Files) passar för FCI eftersom det ger en **delad filresurs (SMB share)** som flera noder kan komma åt. Det fungerar som gemensam lagring för klustret, vilket behövs vid failover.

### Disk Storage → Lagring till VM

Disk Storage passar för VM:ar eftersom det är **blocklagring** med låg latens och hög prestanda. Det används som system- och datadiskar och fungerar som en "riktig hårddisk" för SQL Server i VM.

As mentioned earlier, storage performance is a critical component of an I/O heavy application like a database engine. Azure offers a broad array of storage options and can even build your storage solution to meet your workload requirements.

Azure Storage is a highly scalable, secure storage platform that offers a range of solutions to meet the needs of many applications. Because the focus of this course is databases, you will learn about the aspects of blob storage that are applicable to SQL Server workloads, which are disk, file, and blob storage. You should note that all the above types of storage support encryption at rest with either a Microsoft managed or a user-defined encryption key.

# Azure Managed Disks

- Azure VMs use Managed Disks for their storage
- SQL Server VMs should use either Premium SSD or Ultra Disk for optimal performance

## Azure Managed Disks are offered in four types:

### Ultra Disk

Support high-IO workloads for mission critical databases with extremely low latency

### Premium SSD

High-throughput and low latency and can meet the needs of most database workloads running in the cloud

### Standard SSD

Designed for lightly used dev/test workloads or web servers that do a small amount of IO and require predictable latency

### Standard HDD

Suitable for backups and file storage that is infrequently accessed

© Copyright Microsoft Corporation. All rights reserved.

Här är **grova riktvärden** för **kostnad (kr/månad)** och **latens (ms)** för Azure Managed Disks. Obs: pris beror på **region och diskstorlek**; nedan använder jag vanliga **1 TiB-nivåer** (S30/E30/P30) med publika månadspriser i USD och en ungefärlig växelkurs. ([N2W Software](#))

### Standard HDD (S30, 1 TiB)

**Kostnad:** ca **270 kr/mån** (≈ \$26) ([lucidity.cloud](#))

**Latens:** skriv <10 ms, läs <20 ms (typiskt) ([download.microsoft.com](#))

### Standard SSD (E30, 1 TiB)

**Kostnad:** ca **790 kr/mån** (≈ \$75.19) ([azure.microsoft.com](#))

**Latens:** ofta **lägre än HDD**, mer jämn; tänk **några ms** i praktiken (varierar med I/O-mönster). ([Microsoft Learn](#))

### Premium SSD (P30, 1 TiB)

**Kostnad:** ca **1 520 kr/mån** (≈ \$144.37) ([N2W Software](#))

**Latens:** typiskt **~2–3 ms** ([microsoftpressstore.com](#))

### Ultra Disk (1 TiB som exempel)

**Kostnad:** ~1 290 kr/mån bara för kapaciteten (≈ \$122/mån) + extra för provisionerade **IOPS** och **throughput** (kan bli mycket mer). ([MidForex](#))

**Latens:** **sub-millisekund** (konsekvent låg). ([GitHub](#))

When moving from an on-premises environment, it is recommended to capture metrics **average disk**

**seconds/read** and **average disk seconds/write** from Performance Monitor and **SQL Server:Resource Pool Stats Disk Read and Write IO/sec** to show you how many IOPs SQL Server is serving at its peak.

## Striping disks for maximum throughput



Disks are striped with no redundancy at the operating system level



This allows for you volumes to have the sum of IOPS and data size



For **Premium SSDs** it can be beneficial to scale both IOPs and storage volume



For **Ultra Disk**, you can scale IOPs, throughput, and maximum size independently on a single disk

© Copyright Microsoft Corporation. All rights reserved.

Bilden beskriver **striping av diskar** för att uppnå maximal prestanda, vilket i praktiken motsvarar en **RAID 0**-lösning på operativsystemnivå (t.ex. Storage Spaces i Windows).

Striping innebär att data delas upp och skrivs parallellt över flera diskar. Detta ger **högre throughput och fler IOPS**, eftersom belastningen fördelas på flera enheter.

Nackdelen är att det sker **utan redundans** – om en disk fallerar påverkas hela volymen.

I Azure kan striping vara särskilt användbart när man använder **Premium SSD**, eftersom varje disk har ett maxvärde för IOPS och throughput kopplat till diskstorleken. Genom att kombinera flera diskar kan man "summera" deras kapacitet och nå högre prestanda än en enskild disk klarar. Det kan även vara ett sätt att skala både lagringsvolym och prestanda samtidigt.

För **Ultra Disk** är behovet ofta mindre, eftersom man kan skala **IOPS, throughput och storlek oberoende** på en enda disk. Därför används striping främst när man vill pressa prestanda ur Premium SSD eller när man behöver mycket hög I/O.

- To get optimal performance and larger data volumes and I/O operations per second (IOPS), disks may be striped using Storage Spaces in Windows.
- Since the Azure platform provides storage redundancy, these disks are striped with no redundancy at the operating system level.

One of the ways to get more performance and volume out of Azure disks is to stripe your data across multiple disks. This does not apply to Ultra Disk, as you can scale IOPs, throughput, and maximum size independently on a single disk, however with Premium SSDs it can be beneficial to scale both IOPs and storage volume. In order to stripe disks in Windows, you simply add the number of disks you would like to the VM, and then create a pool using Storage Spaces in Windows. You should not configure any

redundancy for your pool (which would limit your performance) as the redundancy is provided by the Azure framework, which keeps three copies of all disks in synchronous replication, which provides data protection from a disk failure. When you create a pool, your pool has the sum of the IOPs and the volume of your pool. This means if you used 10 P30 disks which are 1 TB and have 5000 IOPs per disk, you would have a 10 TB volume with 50,000 IOPs available.

## SQL Server storage best practices



Create a separate volume for data and transaction log files



Enable read caching on the data file volume



Do not enable any caching on the log file volume



Use the D: drive (the locally attached SSD) for TempDB files



Enable instant file initialization to reduce the impact of file-growth activities

© Copyright Microsoft Corporation. All rights reserved.

Instant File Initialization (IFI) gör att SQL Server kan skapa **datafiler** mycket snabbare (utan att "nollställa" hela filen). Det påverkar främst:

CREATE DATABASE

ALTER DATABASE ... ADD FILE

autogrowth av **datafiler** (.mdf/.ndf)

OBS: IFI gäller **inte** loggfiler (.ldf) – de måste alltid nollställas.

### Så enablar du IFI (Windows + SQL Server)

#### 1) Ge SQL Server-kontot rättigheten "Perform volume maintenance tasks"

Detta är hela "tricket".

Öppna **Local Security Policy**

Kör: secpol.msc

Gå till:

**Local Policies → User Rights Assignment**

Öppna:

**Perform volume maintenance tasks**

Klicka **Add User or Group**

Lägg till SQL Server service account, t.ex.

- NT Service\MSSQLSERVER (default instance)
- NT Service\MSSQL\$INSTANCENAME (named instance)
- eller ett domänkonto / gMSA som SQL Server kör under

OK och stäng.

#### 2) Starta om SQL Server-tjänsten

IFI börjar gälla först efter omstart av SQL Server.

**Kontrollera att IFI är aktivt**

Titta i SQL Server Error Log efter start:

SSMS → **Management** → **SQL Server Logs**

Leta efter en rad liknande:

Database Instant File Initialization: enabled

Om du ser disabled så saknas rättigheten eller så kör SQL Server under ett annat konto än du gav rättigheten till.

**Vanliga frågor****Varför är detta "säkert"?**

Det är en säkerhetsrisk om någon kan läsa råa diskblock och få gamla data. Därför krävs särskild Windows-rättighet.

**Funkar detta i Azure SQL Database?**

Nej – du kan inte styra IFI där. Det gäller främst **SQL Server på Windows**, t.ex. på en Azure VM.

Vill du att jag visar exakt vilket servicekonto din SQL Server kör under (T-SQL + Windows-väg)?

- Plan for an additional 20% of IOPs and throughput when building your storage for your VM to handle workload peaks
- Use the D: drive (the locally attached SSD) for TempDB files because TempDB is recreated upon server restart, so there is no risk of data loss
- Move trace file and error log directories to data disks
- For workloads requiring storage latency under 1ms, you should consider using Ultra Disk over Premium SSD

## Azure SQL VM resource provider

- The SQL VM resource provider can optimize storage configurations tailored to your workload needs.
- You specify the required IOPs and data volume for your database, and the provider will allocate and configure the disks within the operating system.

The screenshot shows the 'Configure storage' page for an Azure SQL VM. It has three tabs: 'General', 'Transactional processing' (selected), and 'Data warehousing'. The page is divided into three main sections: Data storage, Log storage, and TempDb storage.

**Data storage:** This section is for disks attached to the virtual machine as data disks. It includes a 'Data drive location' dropdown set to 'F:\data' and a 'Disk type' dropdown set to 'Premium SSD'. Below this is a table with columns: 'Disk type', 'Size (GiB)', 'Max IOPS', 'Max throughput', and 'Number of disks'. The table shows a configuration for '1024 GiB, Premium SSD' with a size of 1024, 5000 IOPS, 200 MB/s throughput, and 1 disk. A blue information icon and text '1024 GiB, 5000 IOPS, 200 MB/s' are also present.

**Log storage:** This section is for transaction logs. It includes a 'Shared drive space' dropdown set to 'Use a separate drive for log', a 'Log drive location' dropdown set to 'G:\log', and a 'Disk type' dropdown set to 'Premium SSD'. Below this is a similar table with columns: 'Disk type', 'Size (GiB)', 'Max IOPS', 'Max throughput', and 'Number of disks'. The table shows a configuration for '1024 GiB, Premium SSD' with a size of 1024, 5000 IOPS, 200 MB/s throughput, and 1 disk. A blue information icon and text '1024 GiB, 5000 IOPS, 200 MB/s' are also present.

**TempDb storage:** This section is for the TempDb system database. It includes a 'Shared drive space' dropdown set to 'Use local SSD drive' and a 'TempDb drive location' dropdown set to 'D:\tempDb'.

© Copyright Microsoft Corporation. All rights reserved.

One way to reduce the complexity of building storage for your SQL Server VM is to use the SQL Server templates in the Azure Marketplace which allow you to configure your storage as part of your deployment as shown in Figure 23. You can configure the IOPs as need and the template will perform the work of creating your storage spaces pools within Windows.

The above statement will generate a list of T-SQL statements to backup all of the database on the server. Typically, this generated T-SQL will be executed using `sp_executesql` or passed to another program to execute.

# TempDB configuration

## SQL Server uses TempDB for many internal operations

The I/O performance of TempDB can be critical to the performance of the SQL Server

TempDB should have the same number of data files as the server has CPU cores, up to 8

The data files should all be the same size and have the same autogrowth settings

© Copyright Microsoft Corporation. All rights reserved.

SQL Server uses TempDB for many internal operations, in addition to user workloads like temporary tables:

- Using the lowest latency storage possible helps, but it is also important to configure the data files correctly
- In Azure, the best practice is to use D:\ (temporary) drive for TempDB

SQL Server uses TempDB extensively beyond simply storing user defined temporary tables. Work tables that are used to store intermediate query results, sorting operations, and version stores for row versioning are among just a few of the uses for TempDB. Because of this utilization it is important both to place TempDB on the lowest latency storage possible, and properly configure its data files.

Prior to SQL Server 2016, TempDB defaulted to having only one data file. This meant that there could be contention for multiple processes trying to access system pages of the TempDB database (this scenario is fully detailed in Learning Path 5). The common fixes to this scenario were to enable trace flag 1118 which changed the way extents were allocated, and more importantly to create multiple TempDB data files. Because SQL Server uses a proportional fill algorithm for databases with multiple data files, it was also important to ensure that those files were the same size and grew at the same rate. To support this many DBAs used trace flag 1117 which forced all databases with multiple data files, to grow them at the same rate.

## Demo: SQL Server resource provider

**Showcase predefined storage configuration in  
Azure VMs running SQL Server**

© Copyright Microsoft Corporation. All rights reserved.

Demo: Build a SQL Server VM in the Azure Portal, and show the storage configuration settings provided by the SQL Server resource provider

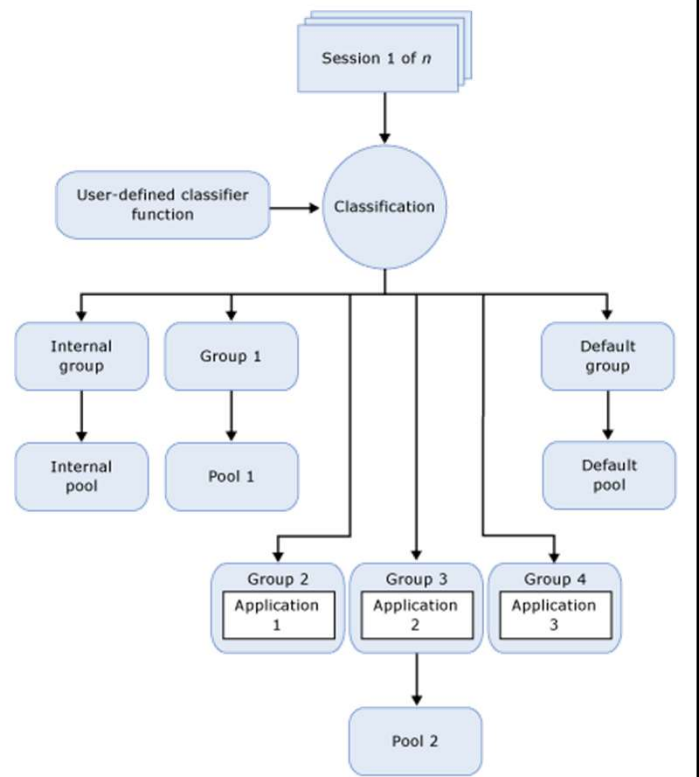
## Resource Governor

- Enables balancing of conflicting workloads with varying resource demands at different times.
- Workloads are segmented into resource pools, enabling you to limit CPU, memory, and IOPs for user sessions.
- Workloads are defined using a classifier function

### Resource Governor components:

- Resource pool
- Workload group
- Classification

© Copyright Microsoft Corporation. All rights reserved.



Resource Governor i SQL Server är en funktion som används för att **styra och balansera resurser** när flera olika typer av arbetslaster körs samtidigt på samma server. Tanken är att undvika att en "tung" workload (t.ex. rapporter eller ETL) tar över hela maskinen och gör att affärskritiska transaktioner blir långsamma. Med Resource Governor kan man dela upp belastningen i separata delar och sätta gränser för hur mycket resurser varje del får använda.

Funktionen bygger på tre huvudkomponenter: **resource pools**, **workload groups** och **classification**. En *resource pool* är en uppsättning resurser (främst CPU och minne) som reserveras eller begränsas. Inom en pool skapar man en eller flera *workload groups* som representerar olika typer av användare eller applikationer, till exempel "App1", "App2" och "Rapporter".

För att placera rätt session i rätt grupp används en **classifier function** (user-defined classifier function). När en ny session ansluter körs funktionen och klassificerar anslutningen baserat på t.ex. login, applikationsnamn eller databas. På så sätt kan man tvinga vissa anslutningar till en begränsad pool och skydda andra från resursbrist.

### Resource Governor in SQL Database and SQL Managed Instance

- Allows to balance conflicting workloads that have different resource demands at different times
- Workloads are divided into resource pools which allow you to limit CPU, memory, and IOPs for user sessions. i.e.:
  - Min/Max CPU Percent
  - Cap of CPU Percent
  - Min/Max Memory Percent
  - NUMA Node Affinity

- Min/Max IOPs Per Volume
- Workloads are classified using a classifier function which can identify the session by metadata and assigns it to a specific resource pool

While some SQL Servers or Azure SQL Managed Instances only support one application's databases (this configuration is commonly seen in mission critical applications), many servers support databases for a multiple applications with differing performance requirements and different peak workload cycles. Balancing these differing requirements can be challenging to the administrator. One of the ways to balance server resources is to use Resource Governor, which was introduced to SQL Server 2008.

Resource governor is a feature in SQL Server and Azure SQL Managed Instance that allows you to granularly control CPU, physical IO, and memory resources that can be used by an incoming application request. Resource Governor is enabled at the database instance level and allows you to define how applications are treated by a classifier function which subdivides sessions into resource pools.

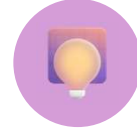
### Resource Governor components

- **Resource pool** represents the physical resources of the server.
- **Workload group** serves as a container for session requests that have similar classification criteria.
- **Classification** process assigns incoming sessions to a workload group based on the characteristics of the session.

Resource Governor is used primarily in multi-tenant scenarios where a group of databases share a single SQL Server instance, and performance needs to be kept consistent for all users of the server. You can also use Resource Governor to limit the resources used by maintenance operations like consistency checks and index rebuilds, to try to guarantee sufficient resources for user queries during your maintenance windows.

The above statement will generate a list of T-SQL statements to backup all of the database on the server. Typically, this generated T-SQL will be executed using `sp_executesql` or passed to another program to execute.

## Knowledge check



Which type of storage should be used in conjunction with Azure VMs for SQL Server data files?

- ☐ Table Storage
- ☐ Blob Storage
- ☒ Disk Storage



Which of the following can be limited using Resource Governor?

- ☐ Buffer Pool Allocation
- ☒ Write IOPs
- ☐ Recompilation



Which is an option from the SQL Server Resource Provider for Azure VMs?

- ☒ Storage configuration
- ☐ Maintenance plan
- ☐ Resource Governor

© Copyright Microsoft Corporation. All rights reserved.

### 1) Which type of storage should be used... for SQL Server data files?

☒ **Disk Storage (rätt)**

SQL Server datafiler på VM ska ligga på Azure Managed Disks.

☒ **Table Storage (fel)**

Table Storage är NoSQL-lagring, inte för SQL Server-databaser.

☒ **Blob Storage (fel)**

Blob passar för backup/filer, inte aktiva MDF/NDF-datafiler.

### 2) Which of the following can be limited using Resource Governor?

☒ **Buffer Pool Allocation (fel)**

Resource Governor kan inte styra buffer pool-minne direkt.

☒ **Write IOPs (fel)**

Resource Governor styr CPU/minne, inte disk-IOPS på detta sätt.

☒ **Recompilation (fel)**

Recompiles påverkas av kod/statistik, inte Resource Governor-regler.

### 3) Which is an option from the SQL Server Resource Provider for Azure VMs?

☒ **Storage configuration (rätt)**

SQL VM resource provider kan automatiskt konfigurera lagring för SQL.

☒ **Maintenance plan (fel)**

Maintenance plans är SQL Server-funktion, inte Azure VM resource provider.

☒ **Resource Governor (fel)**

Resource Governor konfigureras i SQL Server, inte via Azure-provider.

# Configure databases for optimal performance

<https://learn.microsoft.com/training/modules/configure-databases-for-optimal-performance/>

© Copyright Microsoft Corporation. All rights reserved.

In recent versions of SQL Server Microsoft has moved more configuration options to the database level, giving you more granularity in how your databases behave. Along with those options, they have introduced features as part of the intelligent query processing that allow the database optimizer to make better choices when it generates query execution plans.

# Objectives



Understand database scoped configuration options



Understand the features of Intelligent Query Processing (IQP)



Explore the automatic tuning feature in Azure



Understand related maintenance tasks related to indexing and statistics

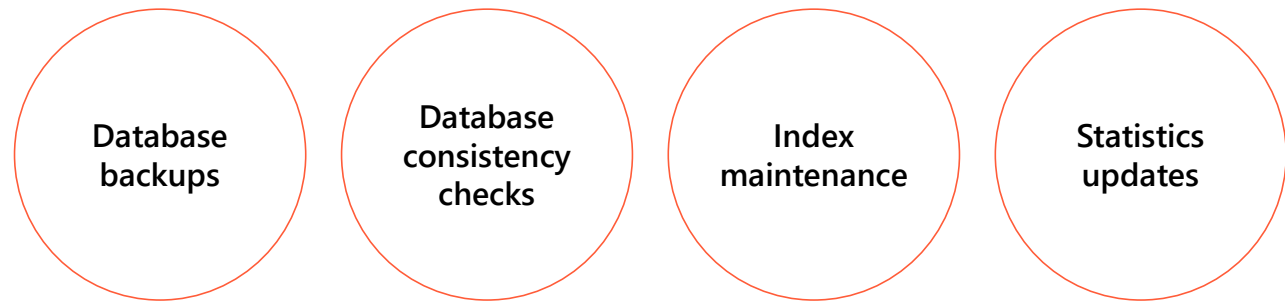
© Copyright Microsoft Corporation. All rights reserved.

In recent versions of SQL Server, Microsoft has moved more configuration options to the database level, giving you more granularity in how your databases behave. Along with those options, they have introduced intelligent query processing features that allow the query optimizer to make better choices.

Even when your database is in the cloud, on-going performance-related maintenance tasks are critical to the overall success of your applications. Whether it's a SQL Server instance in an Azure Virtual Machine or Azure SQL Database, you need to ensure your statistics are current, and your indexes are well organized.

## SQL Server maintenance activities

Databases need regular maintenance activities. Some common examples of these activities include:



© Copyright Microsoft Corporation. All rights reserved.

You learned about the importance of backups and index/statistics maintenance in Learning Paths 4 and 5. Database consistency checks, also known as CHECKDB (for the command DBCC CHECKDB) are of equal importance, as it is the only way to check an entire database for corruption. Depending on the size of your databases and your uptime requirements, you may perform all of these activities nightly. More commonly in production systems, the maintenance operations are spread out over the course of week, as both index maintenance and consistency checks are very I/O intensive operations and they are typically done during weekend hours.

Maintenance plans create a workflow of the tasks to support your databases. Maintenance plans are created as Integration Services packages which allow you to schedule your maintenance activities. Many DBAs also use open source scripts to perform database maintenance, to allow for more flexibility and control of maintenance activities.

## Fragmentation in data files

- ▶ Fragmentation in database files can occur within indexes or tables
- ▶ As data is inserted and updated, records may shift into pages that no longer align with the physical order of data pages on the disk.
- ▶ Fragmentation can degrade the performance of queries
- ▶ Fragmentation can be reduced by **reorganizing** or **rebuilding indexes**

© Copyright Microsoft Corporation. All rights reserved.

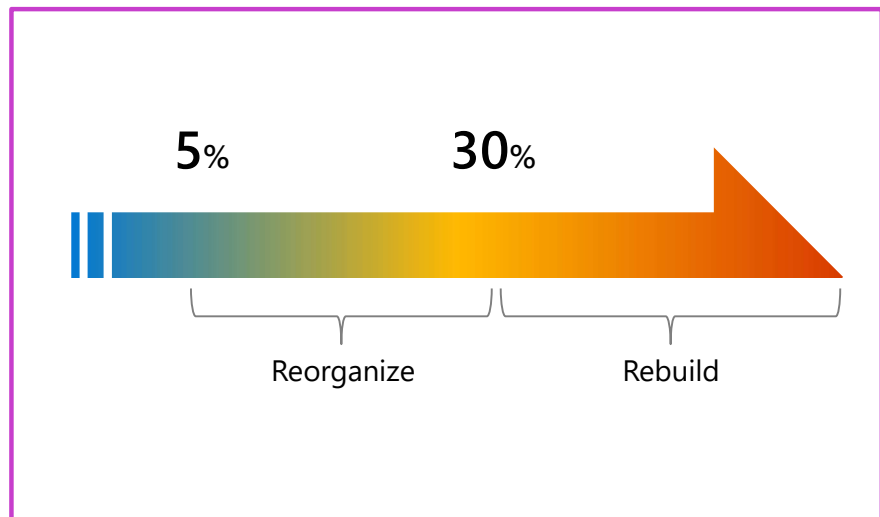
- Fragmentation can degrade the performance of queries, particularly those that perform scan operations
- Fragmentation can be reduced by **reorganizing** or **rebuilding indexes**, and lowering the fill factor of indexes to allow space for updated or inserted records

Fragmentation occurs when indexes (both clustered and nonclustered) have pages in which the logical ordering of the index, which is based on the data value of the index key, does not match the physical ordering of the pages. You will learn more about the structure of indexes in Learning Path 5. Fragmentation is caused by the engine modifying pages whenever insert, update or delete operations take places, and there is no longer space on the page for the new value, causing the page to split. This can degrade performance, particularly of scan operations because additional I/O is required to retrieve data to which the index points. Fragmentation can be reduced by using the fill factor setting in index to allow free space for inserted and updated records.

## Index maintenance tasks

Indexes should be regularly rebuilt and reorganized as they reach defined levels of fragmentation

SQL Server and Azure SQL support resumable index maintenance and online index rebuilds, enabling your workload to continue uninterrupted during maintenance operations.



© Copyright Microsoft Corporation. All rights reserved.

Beyond just proper indexing, index maintenance is an important part of performance, especially for queries which scan tables or index. The query optimizer utilizes statistical information from the indexes to attempt to build the most optimal execution plan. While this execution plan is usually "just good enough" having healthy indexes and statistics will ensure that any given plan will perform at optimal efficiency. Index maintenance should be regularly as data in your databases changes over time. You may choose to change your index maintenance strategy, based on the rate of change of your data.

The common guidance for index maintenance is:

- >5% but < 30%                      Reorganize the index
- > 30%                                      Rebuild the index

You should use this as guidance, but not a hard and fast rule. Depending on your workload and data, you may need to be more aggressive, or in some cases you may be able to defer index maintenance for databases that are mostly perform queries that seek specific pages.

## Index maintenance tasks – REORGANIZE vs. REBUILD

### Reorganize

- Physically reorder the leaf-level index pages to match the logical sorted order of the leaf nodes
- Compacts the index pages based on the index's fill factor setting
- It is an **online** activity

```
ALTER INDEX ALL ON Production.Product
REORGANIZE
```

### Rebuild

- Drops and recreates the pages of the index
- Causes the statistics to be updated
- Can be either online or offline
- When fragmentation is greater than **30%**

```
ALTER INDEX ALL ON Production.Product
REBUILD
```

© Copyright Microsoft Corporation. All rights reserved.

Beyond just proper indexing, index maintenance is an important part of performance, especially for queries which scan tables or index. The query optimizer utilizes statistical information from the indexes to attempt to build the most optimal execution plan. While this execution plan is usually "just good enough" having health indexes and statistics will ensure that any given plan will perform at optimal efficiency. Index maintenance should be regularly as data in your databases changes over time. You may choose to change your index maintenance strategy, based on the rate of change of your data.

The common guidance for index maintenance is:

|               |                      |
|---------------|----------------------|
| >5% but < 30% | Reorganize the index |
| > 30%         | Rebuild the index    |

You should use this as guidance, but not a hard and fast rule. Depending on your workload and data, you may need to be more aggressive, or in some cases you may be able to defer index maintenance for databases that are mostly perform queries that seek specific pages.

## Index maintenance tasks on SQL Server



Since you have the full surface area of Windows and SQL Server (including the SQL Server Agent) you can use the Agent to perform maintenance activities



If you are migrating from on-premises your maintenance activities should be the same as your on-premises workloads



You can use **SQL Agent** or **Task Scheduler** to create custom maintenance jobs

© Copyright Microsoft Corporation. All rights reserved.

With SQL Server being installed within an Azure virtual machine you have access to scheduling services such as the SQL Agent or the Windows Task Scheduler. These automation tools can assist in keeping the amount of fragmentation within indexes to a minimum. With larger databases, a balance between a rebuild and a reorganization of indexes must be found to ensure optimal performance. The flexibility provided by SQL Agent or Task Scheduler allows you to run custom jobs.

You have access to scheduling services such as the SQL Agent or the Windows Task Scheduler. These automation tools can help keeping the amount of fragmentation within indexes to a minimum. With larger databases, a balance between a rebuild and a reorganization of indexes must be found to ensure optimal performance. The flexibility provided by SQL Agent or Task Scheduler allows you to run custom jobs.

## Index maintenance tasks on Azure SQL Database



While the Azure platform handles tasks like consistency checks and backups, the DBA remains responsible for index and statistics maintenance.



Azure SQL Database doesn't have SQL Server Agent, but there are other options for scheduling tasks:

- Azure Automation Runbooks
- Azure SQL Elastic Jobs
- SQL Agent Job from SQL Server in an Azure VM

© Copyright Microsoft Corporation. All rights reserved.

Due to the nature of Azure SQL Database, you do not have access to SQL Server Agent nor Windows Task Scheduler. This means that index maintenance must be controlled from outside of the database. These options will be covered in detail later in this course.

## Index maintenance tasks on Azure SQL Managed Instance



Azure SQL Managed Instance includes the **SQL Agent** and the MSDB database



This means you can use scheduled tasks like on-premises servers or Azure VMs



Any tasks that need to access the file system will need to be modified as that is not supported

© Copyright Microsoft Corporation. All rights reserved.

As with SQL Server on an Azure VM, you can schedule jobs on an Azure SQL Managed Instance via the SQL Server Agent. This will provide flexibility to execute code specifically designed to reduce fragmentation within the indexes in the database

## Statistics maintenance tasks



The database engine uses statistics on columns and indexes to build execution plans



It is important that statistics be kept up to date for all objects



SQL Server defaults to having auto-update statistics on



The database engine uses a sliding scale to determine the number of modifications required to kick off a statistics update

© Copyright Microsoft Corporation. All rights reserved.

## Database scoped configuration options



Databases have two types of configuration options, which are defined based on how they are set:

- Database Scoped Configuration options are configured using ALTER DATABASE SCOPED CONFIGURATION syntax
- Database options are configured using ALTER DATABASE SET syntax



Options include recovery model, isolation level, automatic tuning, and the Intelligent Query Processing features

© Copyright Microsoft Corporation. All rights reserved.

SQL Server has always had several options that were set at the database level. For example, the recovery model has always been a database setting, but as more complex features have been introduced to the database more options have been added. Many of these options are tied to the compatibility level of the database, which is itself a database level configuration option. Database configuration options break down into two groups, with a minor difference:

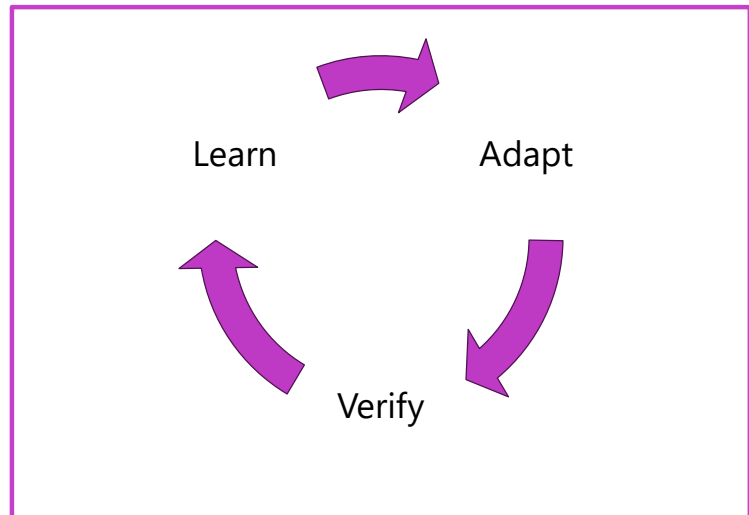
Options configured by the ALTER DATABASE SCOPED CONFIGURATION syntax in T-SQL

Options configured by the ALTER DATABASE SET syntax in T-SQL

There is no significance to the different ways to set these options.

## Automatic tuning

- SQL Server and Azure SQL Database feature automatic tuning that uses the query store as a data source.
- This feature is useful for addressing sudden, unexpected changes in query execution plans that lead to performance regressions.
- Automatic tuning will revert to the last known good execution plan after fifteen executions of a query that has significantly regressed in performance.



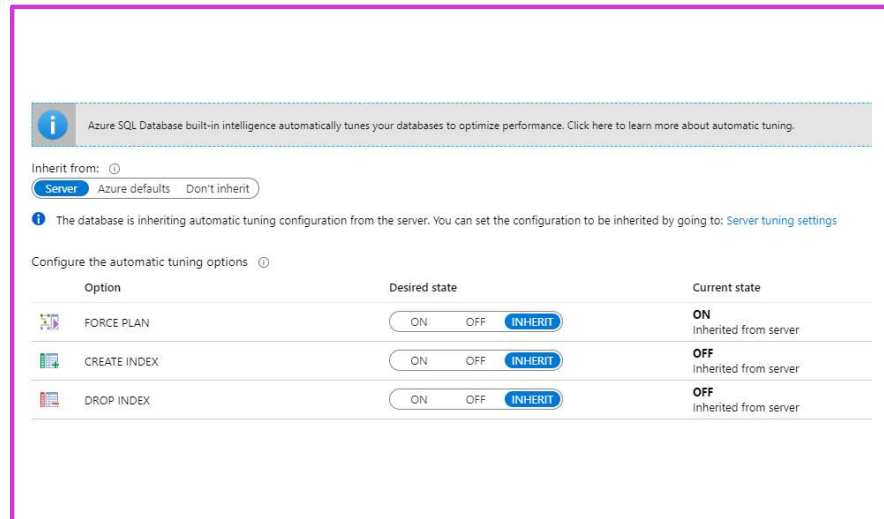
© Copyright Microsoft Corporation. All rights reserved.

Once the feature has been enabled, the database engine will automatically force any recommended query execution plan where the number of errors in the previous plan is higher than the recommended plan, the estimated CPU gain is greater than ten seconds, or a plan was forced and continues to be better than the previous one. The plan will revert back to the last known good plan after fifteen executions of the query.

When plan forcing occurs automatically, the database engine will apply the last known good plan and will also continue to monitor query execution plan performance. If the forced plan does not perform better than the previous plan, it will be subsequently unforced and force a new plan to be compiled. If the forced plan continues to outperform the previously bad plan, it will remain forced until such time a recompile occurs.

# Automatic index management

- Azure SQL Database supports index analysis through automatic index management
- It can add and remove indexes, in addition to forcing a better execution plan in the event of a regression

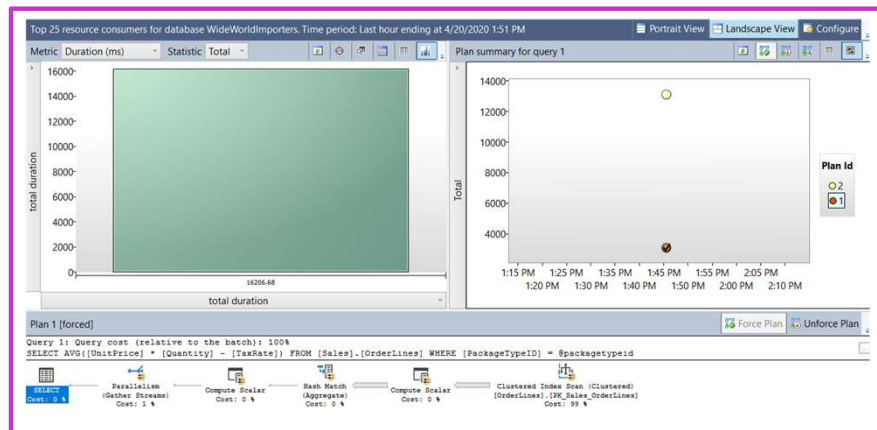


© Copyright Microsoft Corporation. All rights reserved.

As mentioned earlier Azure SQL Database can perform automatic index tuning. Over time, the database will learn about existing workloads and provide recommendations on adding or removing indexes in order to provide better performance. Like forcing improved query plans, the database can be configured to allow for automatic index creation or removal depending on existing index performance, as shown here/

# Automatic plan correction

Identifies when query execution plans have regressed in their performance



```
ALTER DATABASE [<Database Name>] SET AUTOMATIC_TUNING (FORCE_LAST_GOOD_PLAN = ON);
```

© Copyright Microsoft Corporation. All rights reserved.

Automatically forces any recommended query execution plan where:

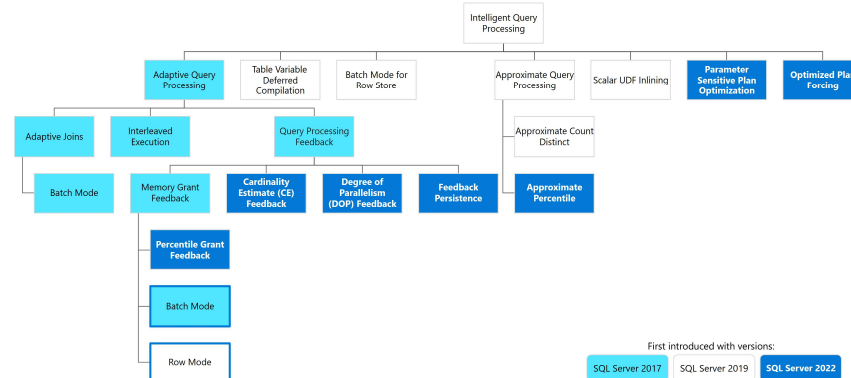
- Number of errors in the previous plan is higher than the recommended plan
- Estimated CPU gain is greater than ten seconds, or
- A plan was forced and continues to be better than the previous one.

If the forced plan does not perform better than the previous plan, it will be unforced and it will force a new plan to be compiled.

## To examine the automatic tuning recommends

*sys.dm\_db\_tuning\_recommendations* (SQL Server 2017 and higher or Azure SQL Database)

# Intelligent Query Processing (IQP)



Intelligent query processing is a family of features that allows the database engine to make better choices to provide better overall performance.

© Copyright Microsoft Corporation. All rights reserved.

Bilden sammanfattar **Intelligent Query Processing (IQP)**, en samling funktioner i SQL Server-motorn som syftar till att automatiskt förbättra frågeprestanda utan att applikationskod behöver ändras. Grundidén är att motorn ska lära sig av faktisk körning och fatta bättre beslut än vad som är möjligt vid enbart kompilering.

En central del är **Adaptive Query Processing**, som gör att exekveringsplaner kan justeras under körning. Hit hör **Adaptive Joins**, där SQL Server kan växla mellan t.ex. nested loops och hash join beroende på verkligt datavolym, samt **Interleaved Execution** för tabellvariabler. **Query Processing Feedback** är en annan viktig del och omfattar bland annat **Memory Grant Feedback**, **Cardinality Estimate (CE) Feedback** och **DOP Feedback**, där motorn justerar minne, uppskattade radantal och parallellism baserat på tidigare körningar. Dessa förbättringar kan dessutom göras bestående via **Feedback Persistence**.

Bilden visar även **Batch Mode on Rowstore**, som ger batch-exekvering även utan columnstore-index, samt **Scalar UDF Inlining**, som eliminerar den traditionella prestandastraffet för skalar-funktioner.

Andra delar är **Parameter Sensitive Plan Optimization** och **Optimized Plan Forcing**, som hanterar parameter sniffing-problem mer intelligent.

**Approximate Query Processing** möjliggör snabba, approximativa beräkningar som APPROX\_COUNT\_DISTINCT.

IQP introducerades stegvis: grunden kom i **SQL Server 2017**, utökades kraftigt i **SQL Server 2019** och vidareutvecklades i **SQL Server 2022**.

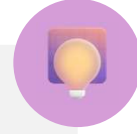
**Azure SQL Database** och **Azure SQL Managed Instance** stöder IQP-funktioner kontinuerligt och ligger ofta i nivå med eller före den senaste SQL Server-versionen.

Most of these features can be implemented with no code change for your applications.

Many of these features correct what were formerly anti-patterns like using user defined scalar value functions and using table variables. These features break down into a few families of features:

- Adaptive Query Processing
- Table Variable Deferred Compilation
- Batch Mode on Rowstore
- Scalar UDF Inlining
- Approximate Query Processing

## Knowledge check



Which intelligent query processing feature allows for faster calculations of a large number of rows?

- ☒ Batch Mode on Row Store
- ☐ Approximate Count Distinct
- ☐ Interleaved Execution



Which component of resource governor allows you to configure limits on system resources?

- ☐ Workload Groups
- ☐ Classifier Functions
- ☒ Resource Pools



You need to migrate a set of databases that use distributed transactions from on-premises SQL Server. Which of the following options should you choose?

- ☒ Azure SQL Managed Instance
- ☐ Azure SQL Database
- ☐ SQL Server in an Azure VM

© Copyright Microsoft Corporation. All rights reserved.

### 1) Faster calculations of a large number of rows?

☒ **Batch Mode on Row Store (rätt)**

Kör batch-bearbetning även på rowstore; snabbare aggregat och join.

☒ **Approximate Count Distinct (fel)**

Ger snabb approximering av COUNT DISTINCT, inte generella beräkningar.

☒ **Interleaved Execution (fel)**

Förbättrar planval för TVF, inte massberäkningar på många rader.

### 2) Resource Governor component for limits on system resources?

☒ **Workload Groups (fel)**

Workload groups organiserar sessions, men gränser sätts via pool.

☒ **Classifier Functions (fel)**

Classifier placerar sessions i grupper, men sätter inga resursgränser.

☒ **Resource Pools (rätt)**

Resource pools definierar CPU/minnesgränser som grupper kan använda.

### 3) Migrate databases using distributed transactions (MSDTC)?

☒ **Azure SQL Managed Instance (rätt)**

Managed Instance har bäst kompatibilitet och stöd för distributed transactions.

☒ **Azure SQL Database (fel)**

SQL Database saknar stöd för MSDTC/distributed transactions mellan databaser.

☒ **SQL Server in an Azure VM (fel)**

VM kan fungera, men MI är PaaS och enklare migration.

## **Instructor led labs: Detect and correct fragmentation issues**

**Investigate index fragmentation**  
**Rebuild fragmented indexes**  
**Validate performance improvements**

© Copyright Microsoft Corporation. All rights reserved.

<https://microsoftlearning.github.io/dp-300-database-administrator/Instructions/Labs/07-detect-correct-fragmentation-issues.html>

## References

### Overview of Azure Monitor

<https://learn.microsoft.com/azure/azure-monitor/overview>

### Monitoring performance by using the Query Store

<https://learn.microsoft.com/sql/relational-databases/performance/monitoring-performance-by-using-the-query-store>

### Overview of Extended Events

<https://learn.microsoft.com/sql/relational-databases/extended-events/extended-events>

### Intelligent Query Processing

<https://learn.microsoft.com/sql/relational-databases/performance/intelligent-query-processing>

© Copyright Microsoft Corporation. All rights reserved.

## Learning Path Recap

### In this learning path, we:

Learned the importance of performance monitoring, including methods to review issues, identify critical Azure metrics, and explore Azure SQL Insights.

You configured SQL Server resources for optimal performance, including Azure Storage options, TempDB data files, and Resource Governor.

Lastly, you learned how to configure databases for peak performance, including database scoped options, maintenance tasks for indexing and statistics, and the auto-tuning feature in Azure.

© Copyright Microsoft Corporation. All rights reserved.



Thank you

© Copyright Microsoft Corporation. All rights reserved.